사이버보안전공 졸업 작품 최종보고서

작품명	눈에 보이는 말				
작품 개요	본 팀의 작품은 수어를 처음 접하는 초보자(청인)들을 위한 '기초 수어 프로그램'입니다. 주기능으로 '한국어를 수어로 번역'해주는 기능과 '수어를 한국어로 번역'이 있고, 부가 기능으로 '사전 기능'이 있습니다. 사용자는 이 기능들을 통해서 이름에 걸맞게 기초 단어들로 이루어진 여러 가지 수어를 직접 배울 수 있습니다.				
팀 명	PsyBer				
학 과	사이버보안전공	학 번	20200713	이 름	이영지
학 과	사이버보안전공	학 번	20200947	이 름	김현진

2023 년 11 월 24 일

지도교수 박 태 정 (서명)

목차

Ⅰ. 설계 목표----------3p
1. 개발 목적
2. 개발 필요성
3. 기대 효과

Ⅱ. 설계 내용----------6p
1. 설계 계획 및 구성도
2. 실제 구현 내용

Ⅲ. 팀 구성 및 역할----------19p

Ⅳ. 개발 일정----------20p

Ⅴ. 결론 및 개선 방향----------21p

Ⅵ. 참고 문헌----------22p

Ⅰ. 설계 목표

1. 개발 목적

의사소통은 사람들 간에 생각이나 감정 등을 교환하는 총체적인 행위로서 사회에서 살아가고 사람들과 관계를 형성하는 데 필수적인 역할을 한다. 하지만, 선천적 또는 후천적 장애로 인해 이러한 가장 기본적인 의사소통에 제약을 받는 사람들이 있다. 바로 청각 장애인과 언어 장애인이다. 이 두 종류의 장애인을 포괄하여 '농인'이라고 부른다[1]. 농인은 상대방의 말을 들을 때 사용하는 청각 기능에 장애가 있거나 상대방에게 말을 전할 때 사용하는 언어 기능에 장애가 있는 사람이다. 그렇다 보니 농인은 청인이 사용하는 것과 다른 언어인 '한국 수어'를 통해 의사소통한다. 그 결과, 농인은 '한국 수어'를 이해하지 못하는 청인과의 의사소통에서 불편함을 느낀다.

2014년 12월 법적으로 농인의 언어인 '한국 수어'를 '한국어'와 동등한 공용어로 인정하는 한국수화언어법이 통과되었다[2]. 공용어임에도 불구하고 청인이 실제로 '한국 수어'를 접할 기회는 매우 적다. 지역 수어 교실 혹은 단기 특강의 형식, 장애인의 날을 기념하여 개최되는 수어 교실 등을 제외하면 청인이 '한국 수어'를 접하기에는 많은 어려움이 있다. 농인은 청인과의 기본적인 소통을 위해 청인의 언어인 '한국어'를 배우려 하지만, 반대로 '한국 수어'를 배우려는 청인은 많지 않다. 본 팀은 이러한 모습을 농인이 청인에게 맞추는, 양방향 적인 것이 아닌 농인의 일방향 적인 노력으로 보았고, 나아가 청인의 노력도 뒷받침될 필요가 있다고 생각했다.

청인이 조금이라도 '한국 수어'에 대한 관심을 가진다면, 농인과 청인은 양방향 적인 노력을 통해 상호 간 의사소통을 더 활발히 할 수 있을 것이다. 따라서, 본 팀은 지능보안프로젝트를 통해 '기초 수어 프로그램'을 개발하고자 한다. 해당 프로그램은 청인을 대상으로 하는 프로그램이다. 앞서 언급한 것처럼 청인이 한국 수어를 접할 기회는 적다. 이에 본 팀은 한국 수어를 배우는 데 있어 '낮은 접근성'을 가장 큰 문제점으로 판단하였고, 이러한 '한국 수어'에 대한 접근성을 높이기 위해 본 프로그램을 개발하고자 한다. '한국 수어'를 편리하게 배울 수 있도록 한다면, 많은 청인이 한국 수어를 배우는 데 거부감도 적고, 언제 어디서든 학습할 수 있을 것으로 기대된다.

2. 개발 필요성

태어난 후 처음 배우는 언어를 흔히 '모국어'라고 한다. 청각, 언어 장애가 없는 청인의 경우, 귀로 들리는 소리를 통해 음성 언어인 '한국어'를 모국어로 습득하게 된다. 이처럼 보통의 한국인의 경우 한국어를 모국어로 배우게 되지만, 모든 한국인의 모국어가 한국어인 것은 아니다. 한국어와 같이 농인에게는 '한국 수어'라는 하나의 언어체계가 존재한다. 선천적 혹은 언어 발달 이전에 청각, 언어 장애를 갖게 된 농인의 경우, 청인처럼 음성 언어를 모국어로 습득하기가 매우 어려워진다. 따라서, 손과 입 등으로 표현하는 '한국 수어'를 모국어로 습득하게 된다[3]. 즉, 농인의 경우 제1 언어가 '한국어'가 아닌 '한국 수어'이며, 한국어를 제1 언어로 소통하는 청인과 같이 농인들은 '한국 수어'를 통해 소통하는 것이다.

청인은 어릴 때부터 음성 언어인 한국어의 발음을 직접 듣고 따라 읽는 연습을 통해 한국어를 듣고, 읽고, 쓸 수 있는 능력을 감추게 된다. 하지만, 한국 수어를 제1 언어로 습득한 뒤 한국어를 배우는 농인은 한국어를 음성 언어로 접할 수 없으므로 완벽한 발음을 구사하기 힘들며 문자를 사용하기에 어려움을 느낀다. 농인의 범주에는 시각적 결함에 대한 것은 없으므로 음성이 아닌 글로 나타내는 것은 충분히 학습할 수 있지 않은가에 대한 의문을 제기할 수 있다. 하지만, 앞서 언급한 것처럼 청인의 언어 학습 과정을 살펴본다면 인간은 말을 먼저 배우고, 이를 문자로 표현하는 과정을 후에 배우는 것으로 언어를 학습한다. 이 과정에서 농인은 전자인 '말'을 구사할 수 없으므로 후자인 '문자'에 대한 학습도 불완전하게 이루어지게 된다. 실제로 농인 167명을 대상으로 한 농인의 문해력 판단 설문조사에서 16명의 청인 중/고등학생의 평균 점수인 16.7점(20점 만점)에 반해 농인은 9.6점으로 낮은 모습을 보였다[3][4].

농인의 의사소통 방법에는 구화, 필담, 보조기기 사용, 수어 이렇게 4가지의 방법이 있다[5]. 하지만, 청각과 언어 기능의 장애 정도에 따라 몇 가지 방법은 제한되기도 한다. 첫 번째로 구화의 경우, 청각 기능의 장애가 심하여 전혀 듣지 못하는 농인은 구화를 통한 대화의 진행이 어렵다. 두 번째 필담의 경우 소수의 대상과 소통에서는 가능할 수 있으나 다수의 대상이 있는 환경에서는 어렵다는 단점이 있다. 또한, 앞서 언급한 것처럼 선천적인 장애가 있는 경우, 문해력이 부족하여 필담을 통한 자유로운 의사소통이 불가능할 수 있다. 세 번째로 청각 장애인은 소리를 듣기 위해 보청기와 보조기기를 사용한다. 하지만, 이 역시 청각 기능의 정도에 따라 사용 여부가 결정된다. 조기 교육을 통해 듣는 방법을 습득한 사람이나 청력 상태가 비교적 좋은 소수에게만 효과를 내는 단점이 있다. 마지막으로 수어는 할 줄 아는 일부 사람에게만 적용되는 제한적인 언어이다. 수어를 배우려면 많은 시간과 노력이 필요하며 개발 목적에서 언급한 것처럼 접근성이 낮아 진입 장벽이 높다. 행사, 교실과 같은 특수한 경우에 수어를 배울 수 있는 환경이 구성되며, 이마저도 다양성이 높지 않아 물리적, 시간적 제약이 많다. 결국, 청인은 수어를 배울 기회가 없기에 수어를 사용하기 어렵고, 수어는 농인 간의 대화에만 유용할 뿐 수어를 모르는 청인과의 의사소통에는 큰 어려움이 있게 된다.

보건복지부의 '장애인 등록 현황'에 따르면 2021년 12월 말 기준 청각 장애인은 약 41만 명, 언어 장애인은 약 2만 3천여 명으로 추정된다[6]. 이는 농인의 수가 총 43만 명에 달한다는 것인데, 이는 결코 적다고 할 수 없을 정도로 많은 수다. 하지만 여전히 사회의 많은 부분에서 청인을 위한 음성 언어인 '한국어'에만 초점을 맞추는 경향이 있다. 이로 인해 농인은 한국어를 배우려고 노력한다. 그렇다고는 하지만, 앞서 언급한 것처럼 농인이 장애의 한계로 인해 한국어를 완벽하게 구사하기는 매우 어렵고, 구화나 필담을 통해 의사소통하더라도 많은 제한이 걸리기 마련이다. 반면에, 청인은 마음만 먹으면 수어를 배울 수 있다. 완벽한 문장 구사까지는 아니더라도 기초적인 단어와 문장을 구사하는 것은 청인에게 어려운 일이 아니다. 우리는 독일어, 프랑스어와 같이 다른 언어와의 의사소통이 필요할 때 각각의 언어를 배우거나 간단한 일상 용어를 배운다. 또한, 사전을 통해 찾아보게 된다. 이처럼 청인이 '한국 수어'를 하나의 언어로 보는 시각을 통해 적어도 일상적인 말을 배우려는 노력을 해야 한다는 것이 본 팀의 생각이다. 이를 가능케 하기 위해선, 다른 언어처럼 '한국 수어'에 대한 교육 방식도 접근성이 높아야 한다. 때문에, 본 팀은 물리적, 시간적 제한 없이 모든 언어의 기본이 되는 인사부터 의사 전달까지 기초적이고 일상적인 한국 수어에 대해 배울 수 있는 프로그램이 필요하다고 생각했고, 구현하기로 하였다.

3. 기대 효과

본 팀이 계속해서 강조하고 있는 바는 '청인이 한국 수어를 배울 기회가 적다'라는 점이다. 기존에 청인이 한국 수어를 배울 수 있는 방법은 다음과 같이 3가지가 있다.

① 한국 수어 교실 및 특강

한국 수어 교실은 전문가와 직접 대면하여 배울 수 있다는 장점이 있다. 하지만, 이러한 수어 교실은 언제 어디에서나 참여할 수 없다는 단점이 있다. 자주 진행되지 않을뿐더러, 인원수 제한도 존재한다. 또한, 참여자는 해당 특강이 열리는 공간에 정해진 시간을 맞춰 가야 한다는 큰 한계점이 존재한다. 결론적으로 이러한 한국 수어 교실은 접근성도 떨어지고, 물리적/시간적 제약이 많다.

② 수어 인터넷 강의

'수어 인터넷 강의'는 EBS에서 제공하고 있어 신뢰성 있고 질 좋은 강의를 무료로 들을 수 있으며, 실생활에서 쓰이는 표현을 주로 배우는 것으로 활용성이 높다[7]. 하지만, 문장 위주의 일상생활 표현으로 수어를 처음 배우는 사람에게는 다소 어려움이 있을 수 있다. 또한, 2006년~2009년에 제작된 오래된 강의이기 때문에 누락되거나 변경된 수어가 존재한다는 문제점이 있다.

③ 수어 학습 프로그램

기존에는 '열린 수어(Open Sign Language)'라는 안드로이드 기반의 수어 학습 프로그램이 존재한다[8]. 해당 프로그램은 수어 사전, 지수어, 수형 사전, 단어장 등을 제공하고 있으며 자신이 원하는 수어를 찾아보고 학습하는 것이 가능하다. 하지만, 해당 프로그램은 국립국어원의 '한국 수어 사전'처럼 사전 기능에 초점을 두었기 때문에 수어를 효과적으로 학습하기에는 다소 어려움이 있다. 우리는 음성 언어를 배울 때 직접 읽고 말하는 연습을 한 뒤, 발음이 틀린 부분에 대해서는 피드백을 받고 이를 고쳐 '완벽'에 이르도록 한다. 수어도 마찬가지다. 수어는 신체를 사용하는 언어이기 때문에[9] 사람의 조건에 따라 한 번에 수어를 제대로 따라 하는 사람이 있을 수 있고, 반대로 설명을 읽은 후에도 제대로 따라 하지 못하는 사람이 있을 수 있다. 이에 해당 프로그램은 설명적인 도움만 줄 뿐, 사용자가 수어를 정확히 하고 있는 게 맞는지에 대한 피드백을 제공해줄 수 없다는 한계점이 있다.

이러한 단점들을 보완하여 본 팀은 청인을 대상으로 한 기초 수어 프로그램을 개발하고자 한다. 해당 프로그램을 통해 기대되는 점은 다음과 같다.

① 청인이 물리적/시간적 제한을 받지 않고 언제 어디서나 쉽게 수어를 접할 수 있다. 사용자가 원할 때 간편하게 수어를 학습할 수 있도록 하는 것에 초점을 두었다. 본 팀의 프로그램은 카메라와 마이크를 필요로 하는데, 현대인의 대부분은 카메라와 마이크를 탑재한 스마트폰을 소지하고 있다. 따라서, 본 팀의 프로그램을 애플리케이션 화한다면 추가적인 장비 없이 스마트폰만으로 언제 어디서든 사용 가능하다는 장점이 있다.

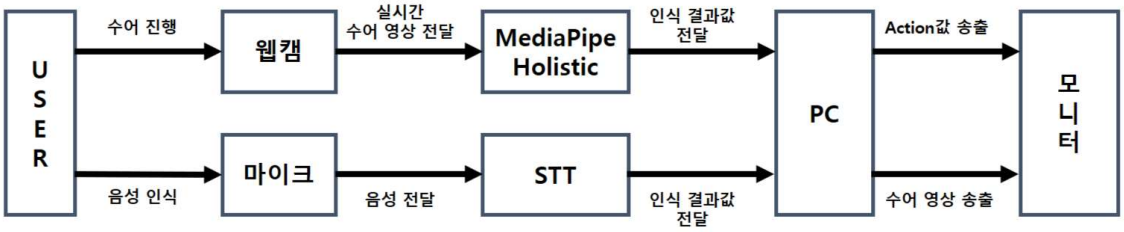
② 기존에 있던 기술들보다 더 편리하게 수어를 배울 수 있다. 기존의 프로그램과 인터넷 강의의 경우, 모두 사전의 성격을 띠고 있다. 본 팀의 프로그램에도 수어 사전 기능이 있으나,

또 다른 기능으로는 한국어 음성인식을 통해 해당 한국어에 맞는 수어 영상을 띄워주는 기능도 있어, 사용자가 하나하나 스크롤을 내리며 찾지 않아도 구두로 간편하게 수어를 찾을 수 있다. 또한, '배고프다'와 '배고파'와 같이 어미가 다르더라도, 혹은 구어체로 편하게 말하더라도 '배고프다'에 대한 수어 영상이 뜨도록 구현하였기 때문에 이 또한 편리함을 극대화해줄 것으로 기대된다.

③ 본 팀의 프로그램은 피드백의 성격도 띠고 있다. 사용자는 다른 기능들을 이용하여 수어 영상과 수형 사진, 수형 설명 등을 통해 수어를 직접 배울 것이다. 하지만, 배운 이후에 본인이 수어를 제대로 하고 있는 것이 맞는지에 대한 의문이 들기 마련이다. 이때, 사용자는 '수어를 한국어로 번역'해주는 기능을 이용할 수 있다. '수어를 한국어로 번역'해주는 기능은 카메라 앞에서 사용자가 수어를 하면, 해당 수어에 맞는 한국어가 뜨도록 구현되어 있다. 때문에, 사용자는 이 기능을 이용하여 수어를 직접 해본 후, 인식이 잘 된다면 수어를 정확히 구사하고 있음을, 인식이 잘되지 않는다면 수어를 정확히 구사하지 못하고 있음을 의미하는 것이므로 사용자에게 피드백을 제공할 것으로 기대된다.

Ⅱ. 설계 내용

1. 설계 계획 및 구성도



본 팀이 설계한 프로그램은 '기초 수어 프로그램'이고, 위의 그림은 본 제품의 시스템 흐름도를 도식화한 그림이다. 이 그림은 본 제품의 주기능인 '수어를 한국어로 번역'해주는 기능과 '한국어를 수어로 번역'해주는 기능의 프로세스를 담고 있다.

그림의 상단 부분은 '수어를 한국어로 번역'해주는 기능의 프로세스이다. 가장 먼저, 사용자가 웹캠 앞에서 수어를 진행한다. 웹캠을 통해 실시간으로 전달된 사용자의 수어 영상은 MediaPipe의 Holistic 기능을 이용하여 사용자의 손과 얼굴, 팔의 랜드마크 진행 과정을 인식한다. 인식된 결과값을 기반으로 해당 수어에 맞는 한국어 결과값(Action값)이 모니터에 실시간으로 송출된다.

그림의 하단 부분은 '한국어를 수어로 번역'해주는 기능의 프로세스이다. 사용자가 마이크에 한국어를 말하면, STT 프로그램을 통해 사용자의 음성을 인식한다. 인식 결과값을 기반으로 해당 한국어에 맞는 수어 영상이 모니터에 송출된다.

마지막으로 도식화에는 포함되지 않았지만, 본 팀이 만든 프로그램에는 '사전 기능'이라는 부가적인 기능도 포함된다. 일반적인 사전처럼 본 팀이 학습시킨 모든 데이터를 사전 목록처럼 나열해 두고, 각각의 단어는 '국립국어원 한국수어사전' 사이트와 연결되어 있다. 따라서, 수어 영상 이외에 수형 사진이나 수형 설명이 필요한 사용자는 사전 기능을 통해 더 구체적이고 자세하게 수어를 배울 수 있다.

< 참고 및 사용한 오픈소스 >

① LSTM 모델과 MediaPipe Holistic 공부를 위해 참고한 오픈소스

본 팀은 프로그램을 구현하기 위해 한 유튜버의 오픈소스를 참고 및 사용하였다. 해당 오픈소스는 2021년 6월 19일, nicknochnack에 의해 GitHub에 공유된 'ActionDetectionforSignLanguage'이다[10]. 해당 오픈소스는 본 팀의 프로그램 취지와 같게 MediaPipe Holistic을 통해 랜드마크를 수집하고, 이를 LSTM을 통해 심층 신경망 학습을 하여 실시간 수어를 탐지하는 프로그램이다. 프로그램 설계 초반에 무엇부터, 어떻게 시작을 해야 할지 감이 오지 않아, 본 팀은 해당 오픈소스의 제작자이자 유튜버인 'Nicholas Renotte'의 'Sign Language Detection using ACTION RECOGNITION with Python | LSTM Deep Learning Model'라는 유튜브 영상을 통해 해당 오픈소스에 대해 설명을 듣고[11], 직접 실습해보면서 LSTM 모델과 MediaPipe Holistc에 대해 공부하는 시간을 가졌다.

② Transformer 모델 공부를 위해 참고한 오픈소스

프로그램 구현에 Transformer 모델을 응용하기 전에, 본 팀은 Transformer 모델에 대한 지식이 전무하였기에 공부가 우선적으로 필요하다고 결론을 내렸다. 이에 기존 LSTM 모델 코드가 Tensorflow 기반이라는 점에 의해 Tensorflow 공식 사이트에서 제공하는 Trasformer 모델에 관한 튜토리얼을 직접 실습해보면서 공부했다[12]. 해당 튜토리얼은 Transformer 모델을 통해 포르투갈어를 영어로 번역해주는 프로그램에 관한 설명이었으며, Transformer에 대한 전반적인 설명도 포함하고 있어 튜토리얼의 프로그램을 완벽히 재현하는 것보다 Transformer가 무엇인지, 코드가 어떤 것이 있는지에 대해 이해하는 것을 중심으로 해당 튜토리얼 오픈소스를 참고하였다.

< 설계에 사용한 주요 기술 : MediaPipe Holistic >

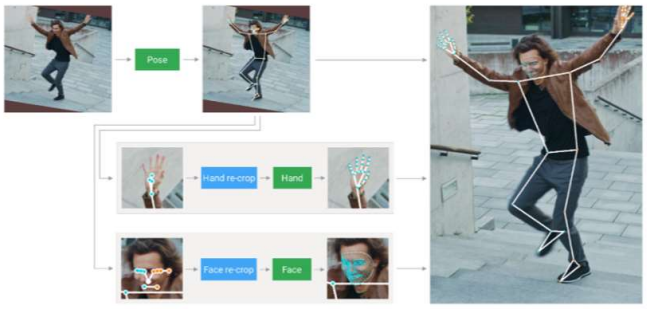


Fig 2. MediaPipe Holistic Pipeline Overview.

본 팀의 프로그램이 '수어'를 인식하는 프로그램이니만큼, 본 팀은 얼굴, 손, 팔 등과 같은 사람의 전반적인 상반신의 스켈레톤 좌푯값을 인식하는 기술을 필요로 했다. 이에 본 팀은 오픈소스 실습을 통해 사용 경험이 있는 MediaPipe의 Holistic을 사용하기로 했다. MediaPipe Holistic은 실시간 3D 객체 탐지 및 포즈, 얼굴, 손의 스켈레톤 좌푯값을 반환해주는 구글 AI 프레임워크다. MeidaPipe Holistic은 MeidaPipe의 'Hands'와 'Face Mesh', 'Pose'를 합친 형태로, Hands 42개, Face 468개, Pose 33개, 총 543개의 랜드마크를 실시간으로 인식한다. 인식은 Pose → Hands → Face 순으로 진행되며, 자세한 인식 과정은 다음과 같다. 가장 먼저, 33개의 랜드마크로 사람의 Pose를 인식한다. 그다음, 손 부분을 재크롭해서 42개의 랜드마크로 사람의 Hands를 인식한다. 마지막으로, 얼굴 부분을 재크롭해서 468개의 랜드마크로 사람의 Face를 인식한다. 본 팀은 MediaPipe이 Holistic과 웹캠을 통해 실시간으로 사용자의 동작을 캡처하여 각각의 랜드마크에 대한 좌푯값을 배열로 변환하여 저장하였다.

```python
mp_holistic = mp.solutions.holistic
mp_drawing = mp.solutions.drawing_utils

def mediapipe_detection(image, model):
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    image.flags.writeable = False
    results = model.process(image)
    image.flags.writeable = True
    image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
    return image, results

def draw_landmarks(image, results):
    mp_drawing.draw_landmarks(image, results.face_landmarks, mp_holistic.FACEMESH_TESSELATION) # Draw face connections
    mp_drawing.draw_landmarks(image, results.pose_landmarks, mp_holistic.POSE_CONNECTIONS) # Draw pose connections
    mp_drawing.draw_landmarks(image, results.left_hand_landmarks, mp_holistic.HAND_CONNECTIONS) # Draw left hand connections
    mp_drawing.draw_landmarks(image, results.right_hand_landmarks, mp_holistic.HAND_CONNECTIONS) # Draw right hand connections

def draw_styled_landmarks(image, results):
    # Draw face connections
    mp_drawing.draw_landmarks(image, results.face_landmarks, mp_holistic.FACEMESH_TESSELATION,
                              mp_drawing.DrawingSpec(color=(80,110,10), thickness=1, circle_radius=1),
                              mp_drawing.DrawingSpec(color=(80,256,121), thickness=1, circle_radius=1)
                              )
    # Draw pose connections
    mp_drawing.draw_landmarks(image, results.pose_landmarks, mp_holistic.POSE_CONNECTIONS,
                              mp_drawing.DrawingSpec(color=(80,22,10), thickness=2, circle_radius=4),
                              mp_drawing.DrawingSpec(color=(80,44,121), thickness=2, circle_radius=2)
                              )
    # Draw left hand connections
    mp_drawing.draw_landmarks(image, results.left_hand_landmarks, mp_holistic.HAND_CONNECTIONS,
                              mp_drawing.DrawingSpec(color=(121,22,76), thickness=2, circle_radius=4),
                              mp_drawing.DrawingSpec(color=(121,44,250), thickness=2, circle_radius=2)
                              )
    # Draw right hand connections
    mp_drawing.draw_landmarks(image, results.right_hand_landmarks, mp_holistic.HAND_CONNECTIONS,
                              mp_drawing.DrawingSpec(color=(245,117,66), thickness=2, circle_radius=4),
                              mp_drawing.DrawingSpec(color=(245,66,230), thickness=2, circle_radius=2)
                              )
```

< MediaPipe Holistic 코드 >

< 학습 모델 >

본 팀은 처음에 LSTM 모델만을 이용하여 프로그램을 구현할 계획이었으나, 교수님들의 피드백을 통해 Transformer 모델로도 프로그램을 구현하기로 하였다. 따라서, 결론적으로 LSTM 모델과 Transformer 모델 각각으로 프로그램을 구현한 후, 두 모델을 비교하여 더 성능이 좋은 것을 전시에 쓰기로 하였다. LSTM과 Transformer에 대한 설명과 비교는 'V. 결론 및 개선 방향'에서 다루었다.

2. 실제 구현 내용

개발 도구	LSTM Model, Transformer Model, STT, MediaPipe Holistic, Jupyter, Visual Studio code, PyQT
개발 언어	Python, Html
개발 장비	앱코 QHD 웹캠 APC930U, 마이크, 모니터

본 팀의 프로그램은 다음 3가지 기능을 가진다. 기능별 구현 내용은 다음과 같다.

< 기초 수어 프로그램의 3가지 기능 >
① 수어를 한국어로 번역해주는 기능
② 한국어를 수어로 번역해주는 기능
③ 사전 기능

2.1 수어를 한국어로 번역해주는 기능

'수어를 한국어로 번역해주는 기능'은 사용자가 웹캠 앞에서 수어를 하면, MediaPipe Holistic을 통해 실시간으로 인식된 랜드마크의 패턴을 기존 학습된 데이터와 비교하여 가장 정확률이 높은 결과값을 화면에 출력해 한국어로 번역해주는 기능이다. 이 기능은 크게 5단계의 구현 과정을 거친다. '데이터 세트 모집 - Extracting - Training - Inference' 순이다. 각 과정 순서별로 구현한 내용은 다음과 같다.

【데이터 세트 모집 단계】

데이터 세트 모집 단계는 프로그램에 학습시킬 수어 단어를 선별하는 단계이다. 프로그램의 취지에 맞게 일상생활에서 많이 사용하는 기초 단어를 위주로 선정하였다. 수어 영상 데이터 세트는 'AIhub'라는 사이트에서 '수어 영상'이라는 데이터 세트를 다운로드 받아 사용하였다[13]. 해당 데이터 세트를 사용하는 데에는 큰 어려움이 있었는데, 이는 'Ⅳ. 개발 일정'에 자세히 작성되어 있다. 결론적으로 해당 데이터 세트에 있는 많은 수어 단어 중, 일상생활에서 많이 쓰거나 사람들이 흥미를 가질만 한 38개의 단어를 선정하였다.

< 선정한 단어(총 38개) >
가다, 감사합니다, 걷다, 괜찮다, 귀엽다, 귀찮다, 나누다, 눕다, 달리다, 덥다, 돕다, 때리다, 맞다, 무섭다, 미치다, 믿다, 바쁘다, 반성하다, 받다, 배고프다, 배부르다, 보이다, 상상하다, 시원하다, 아름답다, 아프다, 안녕하세요, 안다, 예쁘다, 존경하다, 졸리다, 좋다, 죄송합니다, 주다, 춥다, 친하다, 틀리다, 화나다

【Extracting 단계】

설계 초반의 계획은 '데이터 세트 모집' 단계에서 추출한 수어 영상 데이터 세트를 이용하여 수어 모션의 랜드마크를 추출하는 것이었다. 하지만, 이 과정에서 3가지 문제점이 있었고, 해당 문제점은 아래와 같다.

- 문제점 ①. 많은 수의 불필요한 값

수어 영상을 사용하여 랜드마크를 추출할 경우, 추출밖에 불필요한 값들이 많이 포함된다는 문제점이 존재했다. 수어 영상은 하나당 평균적으로 4~5초 정도인데, 이에 비해 실질적으로 수어를 하는 모습은 1~2초에 불과하다. 결과적으로, 수어를 하기 전과 후에 차렷 자세를 유지하고 있는 시간이 약 2~3초에 달한다. 이 영상을 그대로 추출하면, 추출 결과값에 차렷 자세에 대한 추출 값이 절반에 달하게 된다. 이를 학습 데이터로 사용한다면 절반 이상의 불필요한 값들로 인해 생기는 학습의 비효율성과 시간의 비효율적 소요가 발생하게 될 것이라고 판단했다.

- 문제점 ②. OpenCV의 한계

두 번째 문제점은 수어 영상을 OpenCV에 로드하면 영상 속도가 굉장히 느려진다는 점이다. 본 팀이 사용한 MediaPipe Holistic Extracting 코드는 OpenCV를 이용하였는데, OpenCV에 동영상을 로드할 경우, 영상 속도가 실제보다 2~3배는 느려진다는 한계점이 존재했다. 이에 따라 추출 값 또한 2~3배로 늘어난다는 문제점이 발생했다.

- 문제점 ③. 다수의 Nan 값 발생

마지막 문제점은 수어 영상을 로드하여 랜드마크를 추출할 경우, Extracting 과정에서 Nan 값이 다수 발생한다는 점이다. Nan 값이 많이 발생한 영상을 여러 번 다시 추출했음에도 불구하고 계속 동일한 시점에서 Nan 값이 지속적으로 발생하는 것을 확인하였다. 해당 시점을 확인해본 결과, 순간적으로 인식이 풀리는 시점이 원인임을 파악하였다. 수어를 하다 보면 손이 얼굴 앞을 지나가거나 원을 그리는 등의 모션을 하는 경우가 있다. 이런 경우, 손에 의해 얼굴에 대한 인식이 풀리게 되어 그 시점에 대한 얼굴 좌푯값들이 모두 Nan 값으로 뜨게 되어 다수의 Nan 값이 발생하게 된 것이다.

문제점 ①과 문제점 ②를 합쳐보면, 수어 영상 데이터 세트를 이용하여 수어 모션의 랜드마크를 추출할 경우, 추출 값이 실제 필요한 값보다 2~3배로 늘어나는데 그중 절반 이상이 차렷 자세에 대한 불필요한 추출 값이 포함되어 있다는 것을 알 수 있다. 이 문제점들을 이유로, 본 팀은 수어 영상을 이용하는 것이 아닌, 본 팀이 직접 수어를 하여 수어 모션에 대한 랜드마크 값을 추출하기로 하였다. 직접 수어를 하는 것으로 변경되면서, Extracting 코드는 동영상 로드가 아닌 실시간 웹캠을 통한 모션 인식을 하도록 하였으며, 차렷 자세에 대한 불필요한 값을 줄이고 수어에 대한 값만 최대한으로 얻기 위해 frame 수도 조정하였다. 여러 수어를 직접 해본 결과, 거의 모든 수어가 40 frame 안으로 모두 마무리된다고 판단하여 frame 수는 40으로 정하였다.

사실상 Extracting 단계는 Training에 사용할 실질적 데이터 세트를 구성하는 단계다. 총 추출한 수어 단어는 38개이며, 하나의 단어 당 200번의 추출을 하였다. 200번 중 160번은 팀원끼리 각각 80번씩 나누어 직접 수어를 하였고, 나머지 40개는 지원자의 신청을 받아 지원자가 직접 수어를 하였다. 총 8명의 지원자를 받았으며, 각 지원자는 10개의 수어 단어에 대해 20번씩 수어를 하도록 하였다. 이렇게 추출한 200개의 데이터 세트의 구성도는 다음과 같다. 0번부터 79번까지의 데이터 세트는 김현진 학생으로, 80번부터 159번까지의 데이터 세트는 이영지 학생으로 구성이 되어 있다. 이후 40개의 지원자 데이터 세트는 160부터 199까지를

차지한다. 160번부터 179번까지는 지원자1로, 180번부터 199번까지는 지원자2로 구성되어 있다. 데이터 세트의 다양성을 위하여 각 데이터 세트는 다른 날 및 다른 장소, 다른 옷, 다른 거리 등 다양한 환경에서 추출되었다.

```
def extract_keypoints(results):
    pose = np.array([[res.x, res.y, res.z, res.visibility] for res in results.pose_landmarks.landmark]).flatten() if results.pose_landmarks else np.zeros(33*4)
    face = np.array([[res.x, res.y, res.z] for res in results.face_landmarks.landmark]).flatten() if results.face_landmarks else np.zeros(468*3)
    lh = np.array([[res.x, res.y, res.z] for res in results.left_hand_landmarks.landmark]).flatten() if results.left_hand_landmarks else np.zeros(21*3)
    rh = np.array([[res.x, res.y, res.z] for res in results.right_hand_landmarks.landmark]).flatten() if results.right_hand_landmarks else np.zeros(21*3)
    return np.concatenate([pose, face, lh, rh])
```

< Extracting 코드 >

MediaPipe Holistic을 불러와서 실시간으로 인식한 포즈와 얼굴, 손에 대한 좌푯값을 배열로 변환해준다.

【Training 단계】

본 팀은 Training 모델로 LSTM과 Transformer을 사용했다. LSTM 학습은 기존 코드에서 필요한 부분만을 일부 수정하였고[10], Transformer 학습은 기존 코드에서 LSTM 모델에 해당하는 부분을 모두 제거하고 해당 부분에 Transformer 모델의 코드를 추가했다. 참고한 Transformer 오픈소스는 2020년 'HuangWeiKulish'라는 사람에 의해 공유된 'transformer: Positional Encoding.py' 오픈소스를 참고하였다[14].

```
# LSTM Neural Network
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.callbacks import TensorBoard

model = Sequential()
model.add(LSTM(64, return_sequences=True, activation='relu', input_shape=(40,1662)))
model.add(LSTM(128, return_sequences=True, activation='relu'))
model.add(LSTM(64, return_sequences=False, activation='relu'))
model.add(Dense(64, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(actions.shape[0], activation='softmax'))

model.compile(optimizer='Adam', loss='categorical_crossentropy', metrics=['categorical_accuracy'])

model.fit(X_train, y_train, epochs=700)
```

< LSTM Training 코드 >

기존 코드에 있던 LSTM 모델 코드를 그대로 사용하였으며, 일부 하이퍼파라미터 값만 수정하였다.

```
# Transformer Neural Network
def positional_encoding(sequence_length, d_model):
    pos = np.arange(sequence_length)[:, np.newaxis]
    i = np.arange(d_model)[np.newaxis, :]
    angle_rates = 1 / np.power(10000, (2 * (i // 2)) / np.float32(d_model))
    angle_rads = pos * angle_rates
    sines = np.sin(angle_rads[:, 0::2])
    cosines = np.cos(angle_rads[:, 1::2])
    pos_encoding = np.concatenate([sines, cosines], axis=-1)
    return pos_encoding
```

```python
d_model = 1662
num_heads = 4
dense_units = 64

train_samples = X_train.shape[0]
test_samples = X_test.shape[0]

X_train_pe = np.repeat(np.expand_dims(positional_encoding(sequence_length, d_model), axis=0), train_samples, axis=0)
X_test_pe = np.repeat(np.expand_dims(positional_encoding(sequence_length, d_model), axis=0), test_samples, axis=0)


def transformer_model(sequence_length, d_model, num_heads, dense_units, num_classes):
    inputs = Input(shape=(sequence_length, d_model))
    pos_encoding = Input(shape=(sequence_length, d_model))
    attention = tf.keras.layers.MultiHeadAttention(num_heads=num_heads, key_dim=d_model)
    attention_out = attention(inputs, inputs)
    add1 = tf.keras.layers.Add()([inputs, attention_out])
    norm1 = tf.keras.layers.LayerNormalization(epsilon=1e-6)(add1)
    dense1 = tf.keras.layers.Dense(units=dense_units, activation=gelu)(norm1)
    dense2 = tf.keras.layers.Dense(units=d_model, activation=gelu)(dense1)
    add2 = tf.keras.layers.Add()([norm1, dense2])
    norm2 = tf.keras.layers.LayerNormalization(epsilon=1e-6)(add2)
    output = tf.keras.layers.GlobalAveragePooling1D()(norm2)
    output = tf.keras.layers.Dense(units=num_classes, activation='softmax')(output)
    model = Model(inputs=[inputs, pos_encoding], outputs=output)
    return model

model = transformer_model(sequence_length, d_model, num_heads, dense_units, num_classes)
```

< Transformer Training 코드 >

기존 코드에 있던 LSTM 모델 코드를 모두 제거한 후, 해당 부분에 Transformer 모델 코드를 추가하였다.

【Inference 단계】

Inference 단계 또한, LSTM 모델과 Transformer 모델로 각각 실행해보였다. 학습 데이터 세트에 포함되는 본 팀의 구성원들이 직접 수어를 해본 결과, LSTM 모델의 Inference는 약 0.12의 인식률을 보였고, Transformer 모델의 Inference는 약 0.93의 인식률을 보였다.

Transformer 모델의 수어 인식률이 높다는 점을 확인한 후, 본 팀은 전시에 사용할 하나의 모델에만 집중하기로 하였고, 그 결과 Transformer Inference 코드만을 업그레이드하였다. 기존 코드에서 필요한 기능의 코드는 추가하고, 불필요한 기능의 코드는 제거하는 것이 '업그레이드'의 의미다. 기존 코드의 문제점 및 수정해야 할 점은 다음과 같다. 각 문제점을 어떻게 해결했는지 코드와 함께 설명하도록 하겠다.

< 기존 코드의 문제점 및 수정해야 할 점 >

① 38개의 수어에 대한 확률 막대 그래프가 화면에 다 나오지 않음
② 녹화를 멈추는 기능이 없고, 실시간으로 계속해서 입력값을 받음
③ Action 결과값이 한국어가 아닌 영어로 나옴
④ OpenCV 창의 크기가 작음
⑤ Action 결과값의 글씨 크기가 작음

① 38개의 수어에 대한 확률 막대 그래프가 화면에 다 나오지 않음

[문제점] 기존 코드에서는 함수 prob_viz를 이용하여 인식되는 수어의 Action값을 색상 사각형으로 시각화하는 부분이 있었다. 기존 코드를 실습할 때에는 4개의 수어만을 학습시켜 인식하였기 때문에 문제가 되지 않았으나, 본 팀의 경우, 38개의 수어를 학습시키다 보니, 화면의 왼쪽에 38개의 확률 막대 그래프가 다 나오지 않았다. 막대 그래프와 글씨를 줄여

모두 나오게 할 수는 있으나, 이는 오히려 잘 보이지도 않고 미관상 보기 좋지 않다고 판단했다.

[업그레이드 후] 확률 막대 그래프에 대한 코드를 과감하게 제거하기로 하였다.

② 녹화를 멈추는 기능이 없고, 실시간으로 계속해서 입력값을 받음

[문제점] 해당 문제점으로 인해 원하는 결과가 나온 이후에도 다음의 행동을 계속해서 인식하여 너무 많은 결과가 뜨는 결과가 발생했다. 녹화를 멈추지 않고 실시간으로 계속해서 입력값을 바는다면, 쉬지 않고 연속해서 수어를 하고 그에 맞는 결과값을 얻을 수 있다는 장점이 있다. 하지만, 졸업 전시라는 점을 고려해보았을 때, 사용자는 하나의 수어만을 할 것이고, 수어를 했을 때 해당 수어에 맞는 결과값만 떠야 본 팀의 프로그램이 제대로 인식되고 있음을 알 수 있다. 때문에 수어를 한 이후에도 불필요한 행동을 계속해서 인식하고 그에 맞는 결과를 연속해서 송출한다면 이상한 결과값이 화면에 계속 송출되어 사용자는 해당 수어에 대한 인식이 우연히 된 것인지 정상 작동한 것인지 알 수 없게 된다. 또한, 코드가 실행됨과 동시에 바로 인식이 시작되게 되어, 사용자의 준비가 되지 않은 상태에서 갑작스럽게 인식이 시작된다는 문제점도 있다.

[업그레이드 후] ‘Start’와 ‘End’ 버튼을 추가하여 녹화 시간을 임의로 정하도록 하였다. ‘Start’ 버튼을 누르면 녹화가 시작되고, ‘End’ 버튼을 누르면 녹화 중지 및 더 이상의 인식을 하지 않도록 수정하였다. 때문에 사용자는 ‘Start’ 버튼을 누름과 동시에 수어를 하면, 실시간으로 결과값이 뜨게 되며, 프로그램을 멈추고 싶을 때 ‘End’ 버튼을 누르면 마지막 인식된 결과값만 남고, 더 이상의 인식을 하지 않게 된다. 이로써 사용자는 원하는 수어에 대한 결과값만 얻을 수 있다는 장점이 생긴다.

```python
# 마우스 클릭 이벤트 콜백 함수
def mouse_callback(event, x, y, flags, param) :
    global is_recording, recorded_sequence, prev_recording_state

    if event == cv2.EVENT_LBUTTONDOWN :
        if is_inside_button(start_button_coords, x, y):
            is_recording = True
            recorded_sequence = []
        elif is_inside_button(end_button_coords, x, y):
            is_recording = False

# Main loop
with mp_holistic.Holistic(min_detection_confidence=0.5, min_tracking_confidence=0.5) as holistic :
    while cap.isOpened() :
        ret, frame = cap.read()

        # Draw buttons
        cv2.rectangle(frame, start_button_coords[:2], (start_button_coords[0] + start_button_coords[2], start_button_coords[1] + start_button_coords[3]), (0, 255, 0), -1)
        cv2.putText(frame, "Start", (start_button_coords[0] + 30, start_button_coords[1] + 35), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 2)

        cv2.rectangle(frame, end_button_coords[:2], (end_button_coords[0] + end_button_coords[2], end_button_coords[1] + end_button_coords[3]), (0, 0, 255), -1)
        cv2.putText(frame, "End", (end_button_coords[0] + 25, end_button_coords[1] + 35), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2)

        if is_recording :
            image, results = mediapipe_detection(frame, holistic)

            keypoints = extract_keypoints(results)
            recorded_sequence.append(keypoints)
            recorded_sequence = recorded_sequence[-sequence_length:]
```

< Inference 업그레이드 ② 코드 >

③ Action 결과값이 한국어가 아닌 영어로 나옴

[문제점] 기존 코드는 영어 수어를 기반으로 만든 프로그램이기 때문에 결과값이 영어로 나오도록 되어 있다. 하지만, 본 팀은 한국 수어 프로그램이기 때문에 수어를 한 이후, 결과값이 한국어로 나와야 했다. 하지만, OpenCV는 한국어를 지원하지 않는다는 단점이 존재했다. 때문에 각 action에 대해 한국어 mapping을 하더라도 결과값이 '?????'로 뜨고, 정상적인 한국어 결과값이 송출되지 않았다.

[업그레이드 후] 우선, 각 action에 대응하는 한국어로 이루어진 딕셔너리를 만든 후, 입력으로 받은 action값을 딕셔너리에서 찾아 해당 action에 대응하는 한국어 단어를 반환하도록 하였다. 이후에 변환된 한국어 단어를 ImageFont와 ImageDraw 모듈을 사용하여 이미지로서 뜨도록 하였다. 이로써 한국어를 텍스트 그대로 내보내면 결과값이 '????'로 뜨는 OpenCV의 한계를 이미지로 내보내게 함으로써 해결하였다.

```
# 한국어 설정
font = ImageFont.truetype('fonts/gulim.ttc', 25)
img = np.full((200, 300, 3), (255, 255, 255), np.uint8)
img = Image.fromarray(img)
draw = ImageDraw.Draw(img)

action_mapping = {'angry': '화나다', 'beautiful': '아름답다', 'believe': '믿다', 'busy': '바쁘다', 'cold': '춥다',
                  'cool': '시원하다', 'crazy': '미치다', 'cute': '귀엽다', 'divide': '나누다', 'familiar': '친하다',
                  'fine': '괜찮다', 'full': '배부르다', 'give': '주다', 'go': '가다', 'hello': '안녕하세요',
                  'help': '돕다', 'hit': '때리다', 'hot': '덥다', 'hug': '안다', 'hungry': '배고프다',
                  'imagine': '상상하다', 'lie': '눕다', 'like': '좋다', 'pretty': '예쁘다', 'receive': '반성하다',
                  'regret': '받다', 'respect': '존경하다', 'right': '맞다', 'run': '달리다', 'scary': '무섭다',
                  'sick': '아프다', 'sleepy': '졸리다', 'sorry': '죄송합니다','thankyou': '감사합니다', 'tiresome': '귀찮다',
                  'visible':'보이다', 'walk': '걷다', 'wrong': '틀리다'}



# 한국어로 바꿔주는 Code
def translate_action(action):
    korean_action = action_mapping.get(action, '      ')
    return korean_action

# Action 결과값(한국어)
korean_action = translate_action(action)
text = f"Action : {korean_action}"
text_size = draw.textsize(text, font=font)
text_x = 10
text_y = 10
frame_pil = Image.fromarray(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
draw = ImageDraw.Draw(frame_pil)
draw.text((text_x, text_y), text, font=font, fill=(0, 0, 0))
frame = cv2.cvtColor(np.array(frame_pil), cv2.COLOR_RGB2BGR)
```

< Inference 업그레이드 ③ 코드 >

④ OpenCV 창의 크기가 작음

[문제점] 프로그램상 문제가 되진 않지만, 전시라는 특수한 상황을 가정했을 때, 관람자가 보기 편하게 최대한 크게 수정하는 것이 좋다고 생각했다.

[업그레이드 후] OpenCV 창의 크기를 모니터 크기에 맞춰 최대한의 크기로 설정하였다.

⑤ Action 결과값의 글씨 크기가 작음

[문제점] 이것 또한 프로그램상 문제가 되진 않지만, 전시라는 특수한 상황을 가정했을 때, 관람자가 보기 편하게 최대한 크게 수정하는 것이 좋다고 생각했다.

[업그레이드 후] Action 결과값이 잘 보일 수 있도록 글씨 크기를 키웠다.

2.2 한국어를 수어로 번역해주는 기능

'한국어를 수어로 번역해주는 기능'은 사용자가 한국어를 말하면 그에 맞는 수어 영상을 띄워주는 기능이다. 사용자가 마이크에 한국어를 말하면 STT 프로그램을 통해 사용자의 음성을 인식하게 된다. 사전에 수어 영상과 그 의미를 가진 텍스트를 매칭해 놓았기 때문에 음성 인식된 결과를 기반으로 해당 텍스트에 맞는 수어 영상이 송출된다. 여기서의 수어 영상은 '수어를 한국어로 번역해주는 기능'의 구현 단계 중 '데이터 세트 모집' 단계에서 다운로드 받은 '수어 영상' 데이터 세트를 사용하였다.

```
## STT 기능 부분
    # 음성인식
    def start_recognition(self):
        self.audio_label.setText("음성 인식 중입니다...")
        QApplication.processEvents()

        with sr.Microphone() as source:
            audio = self.recognizer.listen(source)

        try:
            text = self.recognizer.recognize_google(audio, language="ko-KR")
            self.audio_label.setPlainText("음성 입력: " + text)
            self.video_file = self.get_video_file(text)
        except sr.UnknownValueError:
            self.audio_label.setPlainText("음성을 인식할 수 없습니다.")
        except sr.RequestError as e:
            self.audio_label.setPlainText("Google API에 접근할 수 없습니다. 오류: {0}".format(e))
```

< STT 코드 ① >

작동 순서는 다음과 같다. 먼저 'Start Recognition' 버튼을 통해 사용자가 원할 때 음성인식 기능이 작동하도록 하였다. 음성이 제대로 인식되었다면 인식된 결과가 뜨고, 반대로 주변의 소음, 마이크의 오작동 등 음성을 인식할 수 없는 상황이라면 "음성을 인식할 수 없습니다."가 뜬다. 또한, 사용자는 'Start Recognition' 버튼을 통해 음성인식을 재시도할 수 있다.

```
if "화나" in text:
    video_file = os.path.join(video_folder, "화나다(angry).mp4")
elif "아름" in text:
    video_file = os.path.join(video_folder, "아름답다(beautiful).mp4")
elif "믿" in text:
    video_file = os.path.join(video_folder, "믿다(believe).mp4")
elif "바쁘" and "바빠" in text :
    video_file = os.path.join(video_folder, "바쁘다(busy).mp4")
elif "춥" in text :
    video_file = os.path.join(video_folder, "춥다(cold).mp4")
elif "시원" in text :
    video_file = os.path.join(video_folder, "시원하다(cool).mp4")
elif "미치" and "미친" and "미쳐" in text :
    video_file = os.path.join(video_folder, "미치다(crazy).mp4")
elif "귀엽" and "귀여" in text :
    video_file = os.path.join(video_folder, "귀엽다(cute).mp4")
elif "나누" and "나눠" in text :
    video_file = os.path.join(video_folder, "나누다(divide).mp4")
elif "친" in text :
    video_file = os.path.join(video_folder, "친하다(familiar).mp4")
elif "괜" in text :
    video_file = os.path.join(video_folder, "괜찮다(fine).mp4")
elif "배부" and "배불러" in text :
    video_file = os.path.join(video_folder, "배부르다(full).mp4")
```

< STT 코드 ② >

텍스트와 매칭되어있는 수어 영상이 있다면 사용자가 'Play Video' 버튼을 눌렀을 때 해당 단어에 맞는 수어 영상이 뜨게 된다. 추가로 본 팀은 사용자가 구어체를 사용할 것을 고려하여 여러 가지 경우의 수를 매칭해 두었다. 예를 들어 학습시킨 단어는 '배부르다'이지만, 구어체 특성상 사용자가 '배부르다'에 비해 '배불러'라고 말할 확률이 높다. 이러한 경우를 대비하여 본 팀은 '배불러'라고 인식되는 경우에도 '배부르다' 영상이 뜰 수 있도록 매칭하였다.

2.3 사전 기능

'사전 기능'은 학습시킨 모든 단어 목록이 일반 사전처럼 나열되어있는 기능이다. 사용자가 목록을 확인하여 원하는 단어를 선택하면 해당 단어에 맞는 '국립국어원 한국 수어 사전' 공식 사이트로 연결된다. 이 기능을 통해 사용자는 언제 어디서든 원하는 수어를 쉽게 찾아볼 수 있으며, 국립국어원에서 제공하는 공식 수어 영상과 수형 사진, 수형 설명을 함께 확인할 수 있다. '한국어를 수어로 번역하는 기능'이 있음에도 불구하고 사전 기능을 제작한 2가지 이유가 있다. 첫 번째 이유는 국립국어원에서 제공하는 수형 사진과 수형 설명은 단지 수어 영상만 제공하는 '한국어를 수어로 번역하는 기능'의 부족함을 채워준다. 또한, 두 번째 이유는 사전 기능을 통해 학습시킨 단어를 직관적으로 볼 수 있다는 점이다. 본 프로그램은 학습시킨 단어에 대해서만 동작한다. '한국어를 수어로 번역하는 기능'은 사용자가 일일이 단어를 찾아보지 않아도 음성인식을 통해 쉽고 간편하게 원하는 단어를 바로 찾을 수 있도록 하기 위한 기능이다. 때문에 사용자는 이 기능을 통해 본 프로그램에 어떤 단어가 학습되어 있는지 직관적으로 알기 어렵고, 학습되지 않은 단어에 대해서는 '해당 영상을 찾을 수 없습니다'라는 답변만 받을 수 있다. 이를 보완하기 위해서 사용자가 꼭 특정한 단어를 궁금해하지 않더라도, 본 프로그램을 통해 배울 수 있는 단어가 무엇인지를 직관적으로 알고, 찾을 수 있도록 '사전 기능'을 부가적으로 구현하였다.

```
self.go.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=11796&top_category=CTE&category=&searc
self.thankyou.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=2372&top_category=CTE&category=&
self.walk.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=3762&top_category=CTE&category=&sear
self.fine.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=9466&top_category=CTE&category=&sear
self.cute.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=28&top_category=CTE&category=&search
self.tiresome.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=4322&top_category=CTE&category=&
self.divide.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=12411&top_category=CTE&category=&s
self.lie.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=12779&top_category=CTE&category=&sear
self.run.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=3903&top_category=CTE&category=&searc
self.hot.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=6492&top_category=CTE&category=&searc
self.help.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=5447&top_category=CTE&category=&sear
self.hit.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=8714&top_category=CTE&category=&searc
self.right.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=11077&top_category=CTE&category=&se
self.scary.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=10967&top_category=CTE&category=&se
self.crazy.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=6548&top_category=CTE&category=&sea
self.believe.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=6700&top_category=CTE&category=&s
self.busy.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=8334&top_category=CTE&category=&sear
self.regret.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=7835&top_category=CTE&category=&se
self.receive.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=23875&top_category=CTE&category=&
self.hungry.clicked.connect(lambda: self.open_link("https://sldict.korean.go.kr/front/sign/signContentsView.do?origin_no=9387&top_category=CTE&category=&se
```

< 사전 코드 >

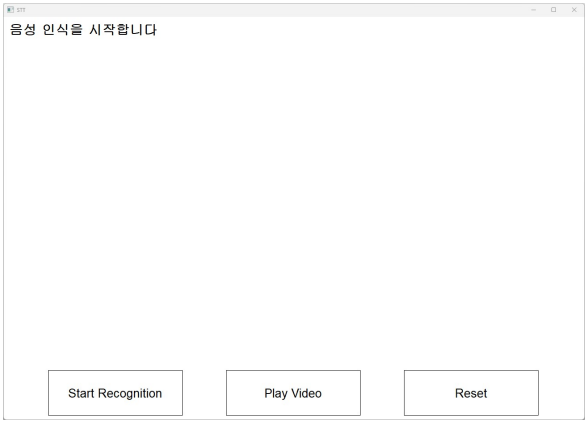
각 단어에 해당하는 국립국어원 공식 사이트 링크를 매칭해둠

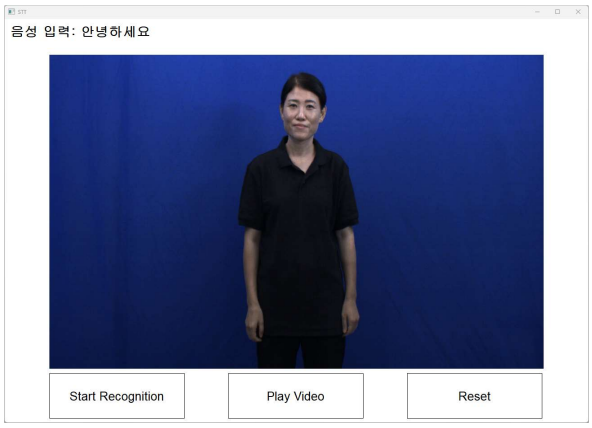
3. 구현 결과

① 프로그램 초기 화면

- ‘한국어를 수어로!’ 버튼 : 한국어를 수어로 번역해주는 기능과 연결된다.
- ‘수어를 한국어로!’ 버튼 : 수어를 한국어로 번역해주는 기능과 연결된다.
- ‘수어 사전’ 버튼 : 부가 기능인 수어 사전 기능과 연결된다.

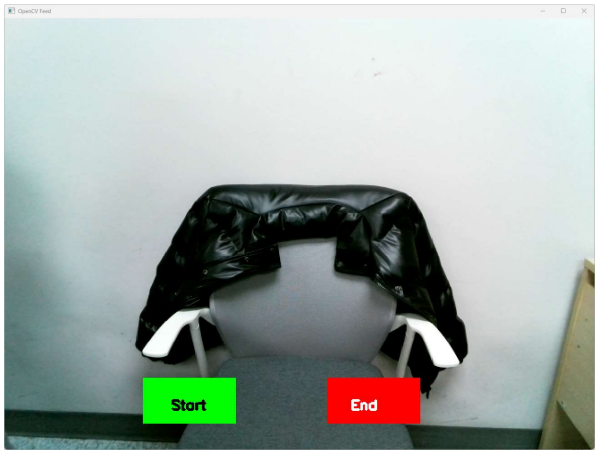
② 한국어를 수어로 번역해주는 기능





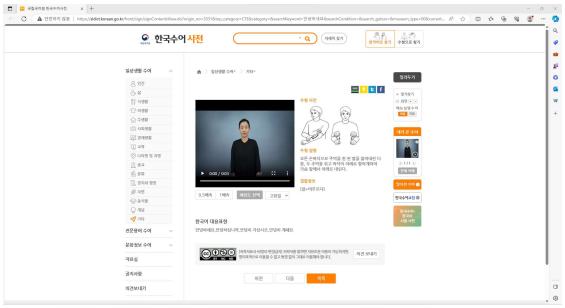
- ‘Start Recognition’ 버튼 : 음성인식이 시작된다.
- ‘Play Video’ 버튼 : 음성인식 결과를 기반으로 해당 결과에 맞는 수어 영상을 재생한다.
  ‘Play Video’ 버튼을 다시 누르면 영상이 처음부터 재생된다.
- ‘Reset’ 버튼 : 처음 화면처럼 음성인식 결과와 영상이 사라진다.

③ 수어를 한국어로 번역해주는 기능



- ‘Start’ 버튼 : 수어 인식이 바로 시작되고, Action 결과값은 왼쪽 상단에 뜬다.
- ‘End’ 버튼 : 수어 녹화와 인식이 중지되고, Action 결과값은 그대로 남아있다.

④ 사전 기능

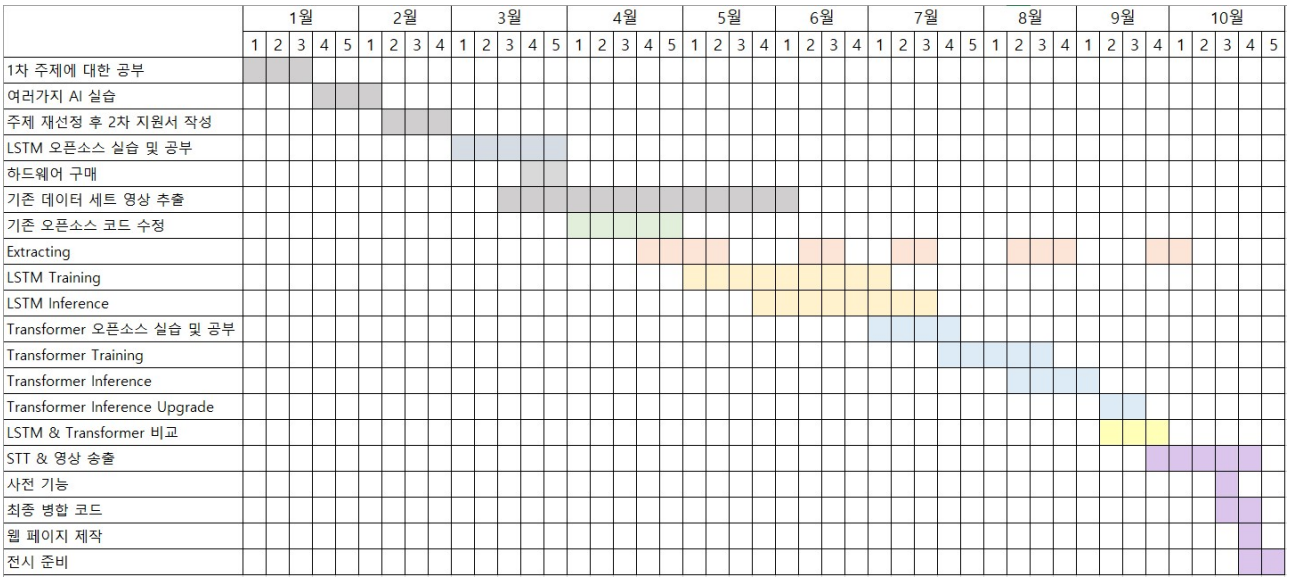


- 학습된 모든 단어가 수어 사전에 뜨며, 원하는 단어를 선택하면 그에 맞는 '한국 수어 사전' 사이트로 연결된다.

Ⅲ. 팀 구성 및 역할

팀 원	역할
김현진	- Extracting - LSTM Training - LSTM Inference - Transformer Training - Transformer Inference - STT & 영상 송출 - 사전 기능 - LSTM & Transformer 비교
이영지	- Extracting - LSTM Training - LSTM Inference - Transformer Training - Transformer Inference - Transformer Inference Upgrade (Upgrade의 의미는 실제 구현 내용에 작성되어 있음) - 웹 페이지 제작 - 사전 기능 - LSTM & Transformer 비교

Ⅳ. 개발 일정



간트 차트에서 볼 수 있듯이, 프로그램 초반에 '기존 데이터 세트 영상 추출'로 인해 많은 일이 지연되었다. 초기 데이터 추출 과정은 다음과 같다. 본 팀은 'AI-HUB'의 수어 영상 데이터 세트를 사용하였다. 해당 데이터 세트는 2.73TB의 수어 영상 데이터로 구성되어 있으며, 자체 저작도구의 내부 편집기를 통해 데이터의 의미를 확인할 수 있었다.

본 팀은 두 가지의 저작도구 중 'Video Annotation Tool'이라는 저작도구를 다운받았다. 해당 도구의 시스템 최소 사양을 맞추기 위하여 본 팀은 지도교수님의 서버 환경에서 작업하였다. 이 도구는 docker를 사용해야 하였는데, docker에 접근하기 위해서는 제작자가 지정한 username과 password가 필요한 상황이었다. 해당 저작도구는 git clone을 통해 다운받을 수 있는 프로그램이었다. 따라서 git 계정에 문제가 있으면 해당 오류가 뜰 수도 있다는 제작자의 의견에 따라 본 팀은 git 계정 확인 및 추가적인 해결 방법을 시도하였지만 해결하지 못하였다. 제작자가 제공한 파일을 통해서 직접 다운을 받는 시도, dev.azure.com에 사용자 등록을 하여 도구를 사용하는 방법도 시도해 보았지만, 문제는 해결되지 않았다. username과 password가 없으면 docker에 접근할 수 없다고 오류 메시지가 떴지만, 제작 측은 애초에 이런 설정을 하지 않았다는 답변을 전해왔기에 본 팀은 데이터를 통해 의미를 확인하는 것이 어려울 것이라고 판단하였다.

또한, 궁극적인 문제는 프로그램에 사용되는 모듈 중 일부의 사양들이 서로 충돌하는 문제였다. 저작도구에서 사용하는 모듈에서 필요로 하는 python의 버전은 지금은 지원하지 않고 사용되지 않는 버전이었다. 이러한 이유로 프로그램 내에서 지속적으로 버전 충돌이 생기는 상황이 이어졌다. 본 팀은 제작 측에서 추가 업데이트를 하지 않는 한 이 도구를 사용하는 것은 어렵다는 결론을 내리게 되었다.

가장 다양한 데이터 세트를 제공하는 곳이었기 때문에, 본 팀은 제작 측에 직접 연락하여 수어의 뜻을 확인할 수 있는 추가적인 파일을 제공받을 수 있는지 문의하였다. 본 팀은 데이터 세트의 사용 의도와 상황을 밝힌 후, 수어 영상의 코드 번호와 의미가 연결된 엑셀 파일을 기밀 유지를 조건으로 제공받아 사용하게 되었다.

Ⅴ. 결론 및 개선 방향

1. 결론

LSTM 모델은 순환 신경망(RNN)의 기법 중 하나이며, 자연어 처리, 음성인식과 같은 시계열 데이터를 주로 처리한다. 또한, 게이트를 추가하여 필요한 기억은 저장하고 필요하지 않은 데이터는 삭제하여 기억하는 정도를 조절하는 것으로 기존 순환 신경망(RNN)의 장기 의존성 문제를 해결한다.

Transformer 모델은 셀프 어텐션(self-attention) 기법을 사용하여 학습 속도를 높이고 메모리 문제를 해결한다. 또한, 입력 시퀀스의 요소들 간의 관계 및 중요성을 파악하여 문맥을 이해하는데, 시퀀스 내의 가까운 것뿐만 아니라 멀리 떨어진 요소들의 관계를 파악하여 장기 의존성을 효과적으로 처리한다. 대표적으로 Chat gpt에서 사용하며 문맥의 이해를 통해 특히 자연어 처리 작업에서 뛰어난 성능을 발휘한다.

< LSTM vs Transformer >

	LSTM	Transformer
loss (평균값)	3.6374	0.0298
accuracy (평균값)	0.0288	0.9947
학습 시간	약 12시간	약 32시간

**여기서의 평균값은 마지막 epoch 100개의 값을 평균화한 값이다.

< LSTM과 Transformer 모델 각각에 대한 다른 사람의 인식 정도 >

우선 LSTM의 경우, 학습 데이터 세트를 구성하고 있는 본 팀의 구성원들이 각 수어 당 5번씩 직접 수어를 해보았지만, 인식률이 거의 0.12에 그쳤다. 따라서, 본 팀은 LSTM 모델을 사용한 '기초 수어 프로그램'의 완전한 구현이 불가능하다고 판단하였다. 반면에 Transformer 모델은 학습 데이터 세트를 구성하고 있는 본 팀의 구성원들에 대해 약 0.93의 인식률을 보였다. 따라서, 본 팀은 학습 데이터 세트에 포함되지 않은 다른 사람들에 대한 인식 정도를 확인하기로 하였다. 지원자는 총 8명의 랜덤한 학생으로 구성되어 있으며, 각 지원자는 이틀에 걸쳐 테스트에 참여하였다. 지원자의 체형 조건 및 옷과 같이 인식에 영향을 줄 수 있는 요소들은 통제하지 않았다. 8명의 지원자는 두 그룹으로 나뉘어 각각 19개의 수어 단어에 대한 인식을 진행하였다. 다음 표는 각 수어 단어에 대해 몇 번 만에 정상적으로 인식이 되었는지를 보여주고 있다. 지원자 8명의 결과값을 평균화하였으며, 소수 첫째 자리에서 반올림하였다.

인식되기까지 시도한 횟수	수어 단어
3번	괜찮다, 바쁘다, 받다, 틀리다
2번	받다, 시원하다, 졸리다
1번	나머지 31개 단어

2. 개선 방향

① 본 팀의 프로그램에는 38개에 대한 기초 수어 단어가 학습되어 있다. 단어의 수를 늘리고, 더 다양한 분야의 단어를 추가한다면 훨씬 완전한 프로그램이 될 것으로 기대된다.

② 본 팀의 프로그램은 기초 수어 '단어'에 초점을 맞춘 프로그램이다. 수어에 대한 지식이 전혀 없는 청인을 대상으로 하는 프로그램이기 때문에 문제가 될 것은 없으나, '문장'에 대한 구현을 하지 않았다는 점은 큰 아쉬움이긴 하다. 추후에 해당 프로그램이 기초적인 수어 '문장'까지 구사할 정도로 업그레이드가 된다면, 사용자는 수어에 대해 더 많은 학습을 할 수 있게 될 것이고, 기존에 있던 프로그램보다 훨씬 더 좋은 수어 교육 프로그램이 될 것으로 기대된다.

③ 앞서 본 팀의 프로그램이 피드백의 성격을 띠고 있다고 언급했다. 하지만, 이는 수어를 제대로 하고 있는 것인지 아닌지에 대한 결과만 알려줄 뿐, 제대로 하고 있지 않은 경우, 무엇이 문제이고 어떻게 수정해야 하는지에 대한 피드백이 제공되지는 않는다. 본 팀의 프로그램에서는 구현하지 못하였지만, 사용자가 수어를 한 후, '왼손바닥을 활짝 펴주세요'나 '손의 위치를 더 내려주세요'와 같이 수어 모션에 대한 구체적인 피드백 설명도 추가로 제공된다면 사용자는 본 프로그램을 통해 더 확실하고 간편하게 수어를 배울 수 있게 될 것으로 기대된다.

Ⅵ. 참고 문헌

[1] 한국수화언어법,(2023.08.08.),「제3조(정의)」국가법령정보센터, https://www.law.go.kr/%EB%B2%95%EB%A0%B9/%ED%95%9C%EA%B5%AD%EC%88%98%ED%99%94%EC%96%B8%EC%96%B4%EB%B2%95 (2023.11.10.)

[2] 한국 수화 언어법, (2016.08.04.), 국회법률정보시스템, https://www.law.go.kr/LSW/lsInfoP.do?lsiSeq=185277&efYd=20160804#0000, (2023.02.20.)

[3] 정희원 & 이승재 & 김아영(국립국어원) & 전북대 21세기 수화 연구단,「수어, 또 하나의 언어」, (2014.04.01.), 국립국어원 (2023.02.15.)

[4] 윤석민(책임연구원), (2014.01.03.) , 농인의 문해 교육 실태 기초 연구, 국립국어원 & 한국농아인협회, (2023.02.23.)

[5] 강희연, "일상 속의 장애인 에티켓 이해하기 - 시각, 청각, 시청각장애인 편", 밀알복지재단, (2021.04), https://miral.org/news/magazine_view.asp?bbsIdx=42922 ,

(2023.11.10.)

[6] 장애인 정책과(박민경) 「장애인 등록 현황(2022년)」, (2023.04.19.), https://www.mohw.go.kr/board.es?mid=a10412000000&bid=0020 , (2023.11.10.)

[7] '함께 배우는 한국 수어' (2009.06.05.~2009.12.26.), EBS, age=3&searchCondition=&searchConditionValue=0&stepId=01BP0PQ000000000037&searchStartDt=&searchEndDt=&orderBy=NEW&searchKeyword=&userId=&searchEndDtValue=0&brdcDsCdFilter=RUN&vodSort=NEW&searchKeywordValue=0&courseId=BP0PQ000000000037&searchStartDtValue=0& (2023.11.10.)

[8] 열린 수어(Open Sign Language) 애플리케이션, https://play.google.com/store/apps/details?id=com.kkomanjell.ksd&hl=ko&gl=US

(2023.11.10.)

[9] 원성옥, 「수화 언어」, (2013.02.25.), https://terms.naver.com/entry.naver?docId=1625339&cid=42251&categoryId=42258 , (2023.11.10.)

[10] Nicknochnack. (2021. 6. 19.). ActionDetectionforSignLanguage. GitHub. https://github.com/nicknochnack/ActionDetectionforSignLanguage

[11] Nicholas Renotte. (2021. 6. 19.). Sign Language Detection using ACTION RECOGNITION with Python | LSTM Deep Learning Model [유튜브 비디오]. 유튜브. https://www.youtube.com/watch?v=doDUihpj6ro

[12] TensorFlow. (2022). Transformer 모델 튜토리얼 [온라인 문서]. TensorFlow 공식 웹사이트. https://www.tensorflow.org/text/tutorials/transformer?hl=ko

[13] Korea Artificial Intelligence Hub. (2022). 수어 영상 [데이터셋]. AIHub. https://aihub.or.kr/aihubdata/data/view.do?currMenu=&topMenu=&aihubDataSe=data&dataSetSn=103

[14] HuangWeiKulish. (2022). Exploratory Data Analysis with Python. GitHub. https://gist.github.com/HuangWeiKulish/b2afa5299aa149507d3f29a4fb1c2e89